Supervision : Comment installer et configurer Nagios Core et Zabbix sur Linux ?

Publié le : 21/03/2026

Auteur : Tom Marchand-Chatelet

Table des matières

Supervision : Comment installer et configurer Nagios Core et Zabbix sur Linux ?....1
I. Présentation....2
II. Installation des serveurs de supervision sur Linux....3
A. Installation de Nagios Core 4.x....3
1. Préparation de l'environnement....3
2. Téléchargement des sources....3
3. Configuration et Création des utilisateurs....3
4. Compilation et Installation des composants....4
5. Configuration d'Apache et accès Web....4
6. Installation des Nagios Plugins (Moteur de vérification)....4
7. Vérification et mise en service....5
B. Installation de Zabbix Server 7.0 (Nginx + PostgreSQL)....7
1. Configuration du dépôt officiel....7
2. Installation des paquets et services....7
3. Préparation de la base de données PostgreSQL....7
4. Importation du schéma initial....7
5. Configuration du serveur Zabbix....8
6. Configuration de Nginx (Gestion de la cohabitation avec Nagios)....8
7. Finalisation et démarrage....8
III. Déploiement de Zabbix Agent 2 (Windows Server)....11
IV. Configuration sur l'Interface Zabbix (Debian)....12
1. Déclaration de l'Hôte....12
2. Liaison du Template....12
3. Test de rupture de service (Simulation de panne)....12
V. Validation du fonctionnement....13
VI. Déploiement de Nagios NSClient++ (Windows)....14
VII. Configuration du Serveur Nagios (Debian)....15
1. Résolution de l'incident "libexec vide"....15
2. Déclaration du serveur dans windows.cfg....15
3. Validation et Redémarrage....15
VIII. Validation du fonctionnement (Nagios)....16
VI. Conclusion....17

I. Présentation

Dans le cadre de l'administration d'un parc informatique, la supervision est un pilier indispensable pour garantir la haute disponibilité des services. Ce tutoriel a pour objectif de vous guider dans la mise en place de deux solutions incontournables : **Nagios Core**, le pionnier historique, et **Zabbix**, la solution moderne et ultra-flexible.

Nous allons voir comment monitorer un serveur **Windows Server 2019** (toujours très présent en entreprise en 2026) ainsi que des hôtes Linux, en configurant des seuils d'alerte et des notifications automatiques.

II. Installation des serveurs de supervision sur Linux

Pour ce TP, nous utiliserons une distribution de type Debian 12 ou Ubuntu 24.04 LTS pour héberger nos services de monitoring.

A. Installation de Nagios Core 4.x

L'installation de Nagios Core sous Linux ne se fait pas via un simple apt install. Pour disposer de la version la plus stable en **mars 2026**, nous compilons les sources manuellement afin de maîtriser l'environnement de sécurité.

1. Préparation de l'environnement

Nous mettons à jour le système et installons les dépendances de compilation, le serveur web Apache et l'utilitaire unzip (indispensable pour l'extraction de certains composants).

```
sudo apt update && sudo apt upgrade -y

sudo apt install -y build-essential apache2 php openssl perl make php-gd libgd-dev libapache2-mod-php libperl-dev libssl-dev daemon wget unzip
```

2. Téléchargement des sources

Nous récupérons l'archive de Nagios Core dans le répertoire temporaire. *Note : Veillez à bien respecter l'ordre des lettres du dossier /tmp.*

```
cd /tmp

wget https://assets.nagios.com/downloads/nagioscore/releases/nagios-4.5.1.tar.gz

tar xzf nagios-4.5.1.tar.gz

cd nagios-4.5.1
```

3. Configuration et Création des utilisateurs

Avant de compiler, il est impératif de générer le Makefile adapté à votre système et de créer les comptes de service.

```
# Préparation de la compilation

sudo ./configure --with-httpd-conf=/etc/apache2/sites-enabled


# Création de l'utilisateur et du groupe nagios

sudo make install-groups-users


# Ajout de l'utilisateur Apache (www-data) au groupe nagios
```

sudo usermod -a -G nagios www-data

4. Compilation et Installation des composants

C'est l'étape où nous transformons le code source en binaires exécutables.

Compilation du programme

sudo make all

Installation des fichiers binaires et de configuration

sudo make install # Fichiers principaux

sudo make install-daemoninit # Scripts de démarrage Systemd

sudo make install-commandmode # Permissions sur les répertoires

sudo make install-config # Fichiers .cfg d'exemple

5. Configuration d'Apache et accès Web

Nous configurons l'interface web et créons le premier compte administrateur.

sudo make install-webconf

sudo a2enmod rewrite cgi

sudo systemctl restart apache2

Création du compte nagiosadmin

sudo htpasswd -c /usr/local/nagios/etc/htpasswd.users nagiosadmin

```
root@SRV-NAGIOSetZabbix:/tmp/nagios-4.5.1# sudo htpasswd -c /usr/local/nagios/etc/htpasswd.users nagiosadmin
New password:
Re-type new password:
Adding password for user nagiosadmin
root@SRV-NAGIOSetZabbix:/tmp/nagios-4.5.1# _
```

6. Installation des Nagios Plugins (Moteur de vérification)

Sans les plugins, Nagios ne peut effectuer aucun test (Ping, CPU, etc.).

cd /tmp

wget https://nagios-plugins.org/download/nagios-plugins-2.4.6.tar.gz

tar xzf nagios-plugins-2.4.6.tar.gz

cd nagios-plugins-2.4.6

Compilation et installation des plugins

sudo ./configure

sudo make

```
sudo make install
```

7. Vérification et mise en service

Avant de démarrer, nous effectuons un "Pre-flight check" pour valider la syntaxe des fichiers de configuration.

sudo /usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg

```
root@SRV-NAGIOSetZabbix:/tmp/nagios-4.5.1# sudo /usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg

Nagios Core 4.5.1
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
Last Modified: 2024-02-28
License: GPL

Website: https://www.nagios.org
Reading configuration data...
   Read main config file okay...
   Read object config files okay...

Running pre-flight check on configuration data...

Checking objects...
        Checked 8 services.
        Checked 1 hosts.
        Checked 1 host groups.
        Checked 0 service groups.
        Checked 1 contacts.
        Checked 1 contact groups.
        Checked 24 commands.
        Checked 5 time periods.
        Checked 0 host escalations.
        Checked 0 service escalations.
Checking for circular paths...
        Checked 1 hosts
        Checked 0 service dependencies
        Checked 0 host dependencies
        Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

Total Warnings: 0
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check
root@SRV-NAGIOSetZabbix:/tmp/nagios-4.5.1# _
```

Si le rapport indique **Total Errors: 0**, lancez le service :

sudo systemctl start nagios

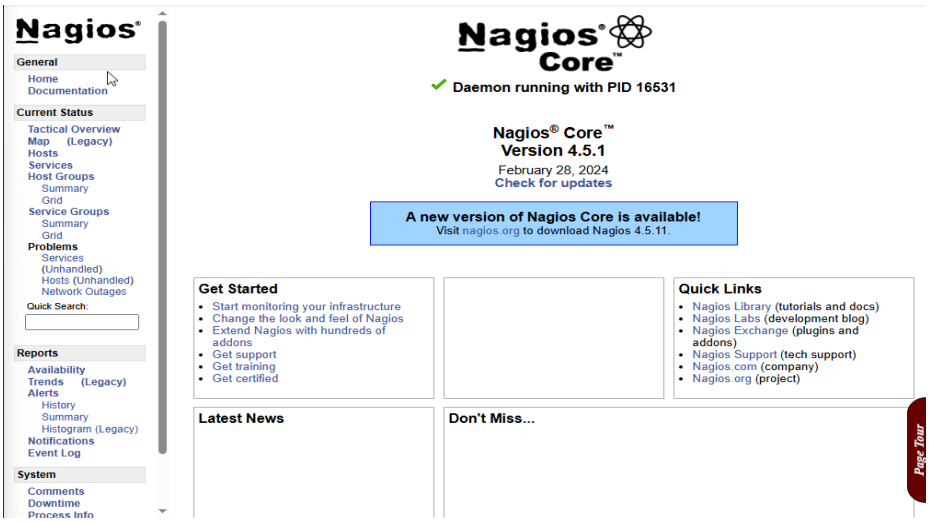
sudo systemctl enable nagios

Accès à l'interface

L'interface de supervision est désormais accessible à l'adresse suivante : http://<IP_DE_VOTRE_SERVEUR>/nagios

Connectez-vous avec :

- **Identifiant :** nagiosadmin
- **Mot de passe :** défini à l'étape 5.



B. Installation de Zabbix Server 7.0 (Nginx + PostgreSQL)

Contrairement à Nagios, Zabbix s'appuie massivement sur une base de données relationnelle pour stocker l'historique des métriques. Nous utilisons ici **PostgreSQL**, choisi pour sa robustesse et ses performances avec les gros volumes de données en 2026.

1. Configuration du dépôt officiel

Pour obtenir la version 7.0 (LTS), nous utilisons le dépôt officiel de Zabbix plutôt que les dépôts standards de Debian.

```
wget https://repo.zabbix.com/zabbix/7.0/debian/pool/main/z/zabbix-release/zabbix-release_latest+debian12_all.deb

sudo dpkg -i zabbix-release_latest+debian12_all.deb

sudo apt update
```

2. Installation des paquets et services

Nous installons le serveur, l'interface web, le moteur PostgreSQL et les scripts de base de données nécessaires. *Note : Sur Debian 12, nous utilisons la version stable **PHP 8.2**.*

```
sudo apt install postgresql postgresql-contrib zabbix-server-pgsql zabbix-frontend-php php8.2-pgsql zabbix-nginx-conf zabbix-sql-scripts zabbix-agent -y
```

3. Préparation de la base de données PostgreSQL

Il faut créer un utilisateur dédié et une base de données. *Attention : Veillez à bien orthographier "zabbix" pour éviter toute erreur d'importation.*

```
# Connexion à l'instance PostgreSQL

sudo -u postgres psql


# Commandes SQL à exécuter :

CREATE USER zabbix WITH PASSWORD '1234';

CREATE DATABASE zabbix OWNER zabbix;

\q
```

4. Importation du schéma initial

Cette étape injecte la structure des tables (plusieurs centaines) dans la base de données. Elle peut prendre environ 30 secondes.

```
zcat /usr/share/zabbix-sql-scripts/postgresql/server.sql.gz | sudo -u zabbix psql zabbix
```

5. Configuration du serveur Zabbix

On indique au service Zabbix comment s'authentifier auprès de la base de données.

```
sudo nano /etc/zabbix/zabbix_server.conf
```

Cherchez et modifiez la ligne suivante (pensez à enlever le # au début) :
DBPassword=1234

6. Configuration de Nginx (Gestion de la cohabitation avec Nagios)

Nagios occupant déjà le port **80** avec Apache, nous configurons Nginx pour écouter sur le port **8080**.

```
sudo nano /etc/zabbix/nginx.conf
```

Modifiez les lignes suivantes :

```
listen 8080;

server_name 192.168.1.112; # L'IP de votre serveur Debian
```

Astuce : Pensez à supprimer le site par défaut d'Nginx pour éviter tout conflit : sudo rm /etc/nginx/sites-enabled/default.

7. Finalisation et démarrage

On active et redémarre l'ensemble des services pour valider la configuration.

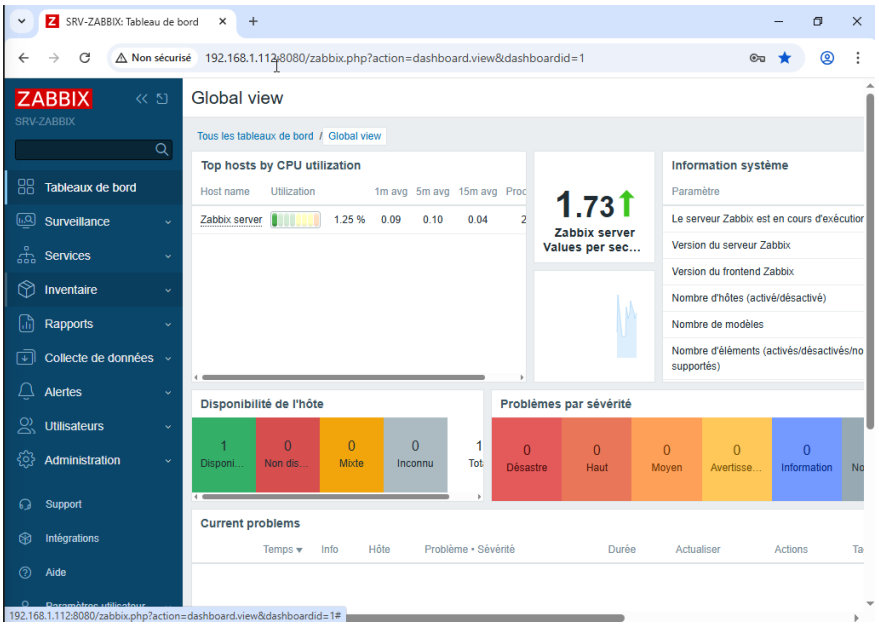
```
sudo systemctl restart zabbix-server zabbix-agent nginx php8.2-fpm postgresql

sudo systemctl enable zabbix-server zabbix-agent nginx php8.2-fpm postgresql
```

Accès à l'interface Web

L'interface de Zabbix est disponible à l'adresse : http://192.168.1.112:8080

Étapes de l'assistant (Setup) :

1. **Vérification des prérequis :** Tout doit être sur "OK".
2. **Base de données :** Database zabbix, User zabbix, Password 1234.
3. **Timezone :** Sélectionnez Europe/Paris.
4. **Connexion :** Identifiants par défaut :
    - **Utilisateur :** Admin (avec un **A** majuscule)
    - **Mot de passe :** zabbix



III. Déploiement de Zabbix Agent 2 (Windows Server)

L'installation via **PowerShell** a été privilégiée pour automatiser le processus sur le serveur **192.168.1.103**.

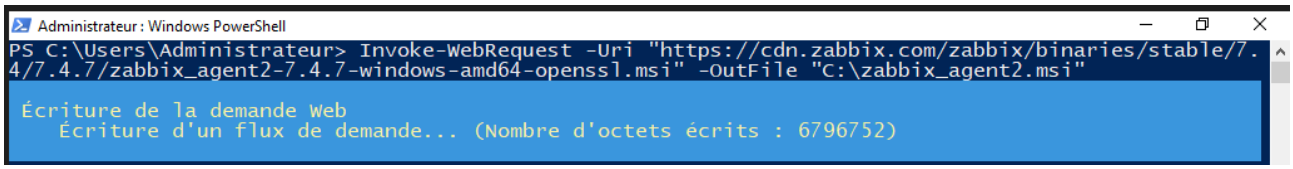
Ouverture du Pare-feu (PowerShell) :

New-NetFirewallRule -DisplayName "Zabbix Agent 2" -Direction Inbound -Action Allow -Protocol TCP -LocalPort 10050

Téléchargement et Installation

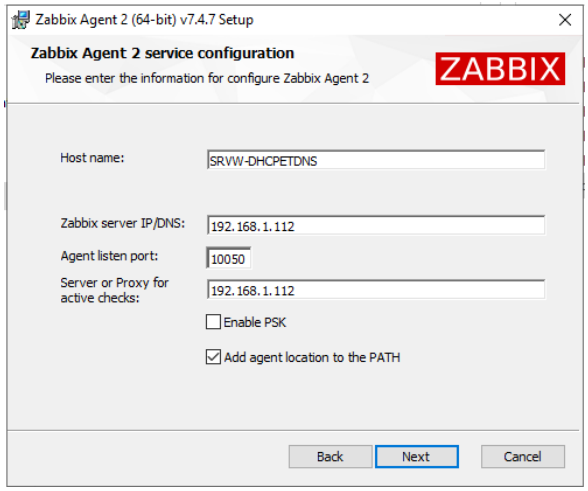
Téléchargement du MSI

Invoke-WebRequest -Uri "https://cdn.zabbix.com/zabbix/binaries/stable/7.4/7.4.7/zabbix_agent2-7.4.7-windows-amd64-openssl.msi" -OutFile "C:\zabbix_agent2.msi"



Paramètres critiques saisis dans l'assistant :

- **Host name :** SRVW-DHCPETDNS (Identification unique).
- **Zabbix server IP (Passif) :** 192.168.1.112.
- **ServerActive (Actif) :** 192.168.1.112.
- **Port :** 10050.



Note : L'option "Add agent location to the PATH" a été cochée pour faciliter le diagnostic en ligne de commande.

IV. Configuration sur l'Interface Zabbix (Debian)

La configuration repose sur l'utilisation de **Templates** pour automatiser la création des indicateurs.

1. Déclaration de l'Hôte

Rendez-vous dans **Data Collection > Hosts > Create host** :

- **Nom de l'hôte :** SRVW-DHCPETDNS (doit être identique à la config agent).
- **Groupe :** Windows servers.
- **Interface Agent :** IP 192.168.1.103 | Port 10050.

2. Liaison du Template

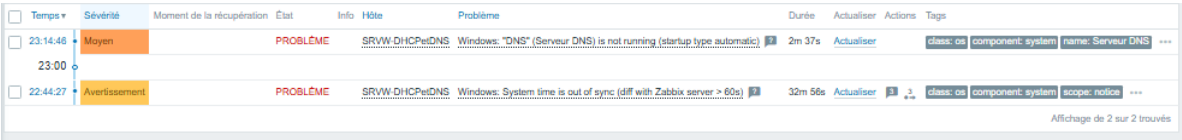
Nous avons lié le modèle officiel : Windows by Zabbix agent.

- **Items :** +100 points de contrôle (CPU, RAM, Disques).
- **Triggers :** Seuils d'alerte prédéfinis (ex: disque plein à 90%).
- **LLD (Discovery) :** Détection automatique des services **DHCP** et **DNS**.

3. Test de rupture de service (Simulation de panne)

Afin de vérifier la pertinence des alertes, nous avons simulé une panne critique sur le serveur Windows (192.168.1.103).

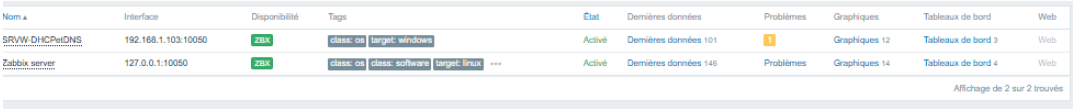
- **Action** : Arrêt manuel du service **DNS** via la console des services Windows.
- **Constat dans Zabbix** : Le déclencheur (*Trigger*) a détecté l'arrêt en moins de deux minutes. Une alerte de sévérité "Average" est apparue sur le tableau de bord avec le message : Windows: "DNS" (Serveur DNS) is not running.

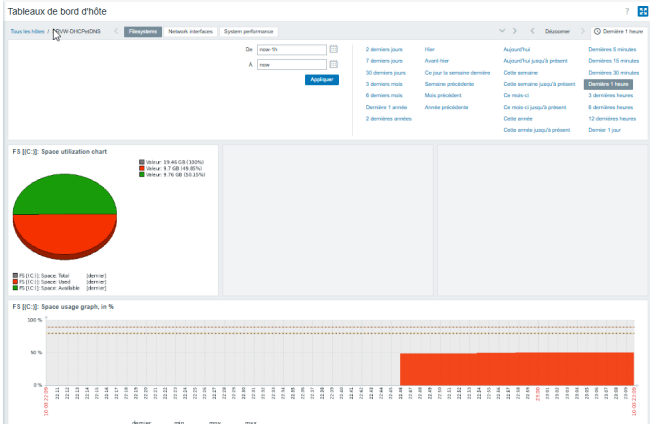


V. Validation du fonctionnement

La réussite de l'installation est confirmée par deux indicateurs :

1. **Disponibilité :** L'icône **ZBX** passe au **Vert** dans la liste des hôtes.
2. **Flux de données :** Dans **Monitoring > Latest data**, les métriques (charge CPU, RAM libre) remontent en temps réel.





VI. Déploiement de Nagios NSClient++ (Windows)

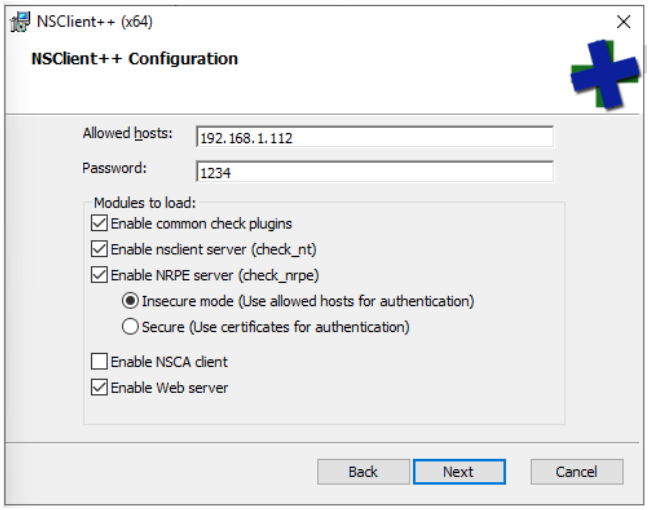
L'agent **NSClient++** permet à Nagios d'interroger les compteurs de performance Windows (CPU, RAM, Disques) via le protocole check_nt sur le port **12489**.

Installation et Configuration de l'Agent

Le package MSI a été récupéré sur le dépôt officiel GitHub.

Paramètres critiques lors de l'installation :

- **Allowed hosts** : 192.168.1.112 (IP du serveur Nagios).
- **NSClient Password** : 1234 (Code d'authentification pour les requêtes).
- **Modules activés (Crucial)** :
  - Enable common check plugins
  - Enable nsclient server (check_nt) **<-- Indispensable pour le port 12489.**
  - Enable NRPE server (en mode *Insecure* pour le TP).



Ouverture du Pare-feu (PowerShell) :

```
New-NetFirewallRule -DisplayName "Nagios NSClient" -Direction Inbound -Action Allow -Protocol TCP
-LocalPort 12489
```

VII. Configuration du Serveur Nagios (Debian)

Pour que Nagios puisse interroger l'agent, nous avons dû lier le moteur aux plugins et déclarer l'hôte.

1. Résolution de l'incident "libexec vide"

Lors de l'installation manuelle de Nagios Core, le répertoire des plugins (/usr/local/nagios/libexec/) était vide. Nous avons corrigé cela par l'installation des paquets système et la création de liens symboliques :

```
sudo apt install monitoring-plugins-basic -y

sudo ln -s /usr/lib/nagios/plugins/* /usr/local/nagios/libexec/
```

2. Déclaration du serveur dans windows.cfg

Édition du fichier /usr/local/nagios/etc/objects/windows.cfg pour définir l'hôte et son service d'Uptime :

Extrait de code

```
define host {
    use             windows-server
    host_name       winserver
    alias           SRVW-DHCPETDNS
    address         192.168.1.103
}

define service {
    use             generic-service
    host_name       winserver
    service_description Uptime
    check_command   check_nt!UPTIME!-s 1234
}
```

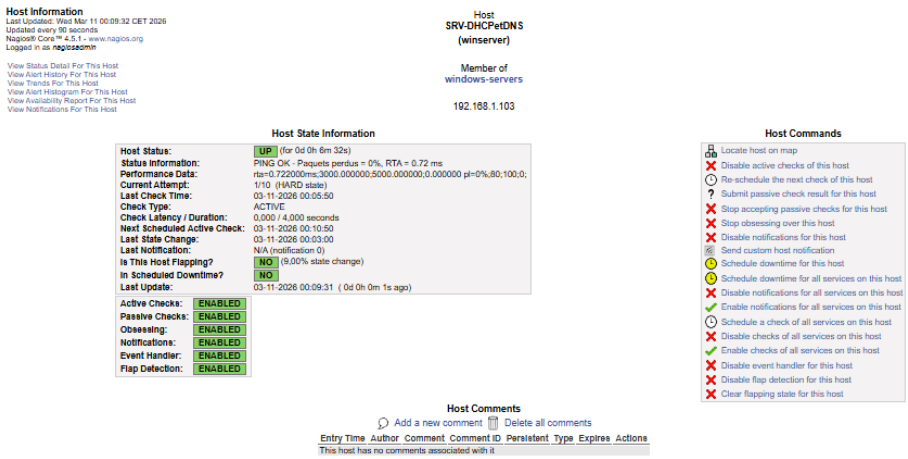
3. Validation et Redémarrage

```
# Vérification de la syntaxe
sudo /usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg
# Application
sudo systemctl restart nagios
```

VIII. Validation du fonctionnement (Nagios)

La réussite de l'installation et de la liaison avec l'agent NSClient++ est confirmée par deux indicateurs :

- **Disponibilité de l'hôte** : Dans le menu **Hosts**, le serveur winserver affiche un statut **UP** (Vert). Le "Status Information" indique un PING OK, confirmant la connectivité réseau.
- **Remontée des services** : Dans le menu **Services**, les indicateurs (Uptime, CPU, RAM) affichent un statut **OK**. Cela valide que Nagios parvient à interroger l'agent Windows via le port **12489** avec le mot de passe configuré.



VI. Conclusion

Vous avez maintenant une plateforme de supervision opérationnelle pour votre infrastructure SISR. Nagios restera votre outil robuste pour les vérifications simples et scriptées, tandis que Zabbix vous offrira une visibilité graphique et une gestion par templates beaucoup plus souple pour vos serveurs Windows Server 2019.

N'oubliez pas de tester vos alertes en simulant une panne (arrêt d'un service ou saturation artificielle d'un disque) pour vérifier la bonne réception des notifications !